

Análisis Teórico y Experimental de Algoritmos de Divide y Vencerás: Ordenamiento, Búsqueda y Selección

Andres Barbosa[†], Emmanuel Velásquez[†] y Diego Oyarzo[†]

[†]Universidad de Magallanes

Este informe fue compilado el 14 de mayo de 2026

Resumen

El presente proyecto analiza empírica y teóricamente el comportamiento de algoritmos basados en el paradigma de Divide y Vencerás, aplicándolos sobre un robusto sistema de gestión de deportistas construido nativamente en C. El estudio abarca algoritmos de ordenamiento como Merge Sort (en su vertiente clásica y optimizada mediante Insertion Sort) y Quick Sort utilizando el esquema particional de Lomuto con 4 heurísticas de pivote. Asimismo, se incorporan esquemas de Búsqueda Binaria (recursiva y delimitación de rangos), Búsqueda Exponencial y Búsqueda por Interpolación; cerrando el abanico con Quick Select para la búsqueda del k-ésimo elemento.

Mediante evaluaciones sobre variados tamaños de entrada y arreglos dispares, los resultados arrojan evidencia rigurosa sobre el salto cualitativo en la complejidad temporal asintótica (de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$) en comparación a técnicas sistemáticas previas y explora en profundidad cómo la selección paramétrica del pivote o los umbrales de combinación moldean determinadamente el rendimiento final en escenarios prácticos.

Keywords: Divide y Vencerás, Merge Sort, Quick Sort, Búsqueda Exponencial, Quick Select, Complejidad Temporal, Lenguaje C, Lomuto

■ Índice

1	Introducción	3
2	Objetivo principal	3
2.1	Objetivos secundarios	3
3	Desarrollo	4
3.1	Análisis Teórico de Algoritmos de Ordenamiento	4
	Merge Sort: Clásico y Optimizado • Quick Sort y sus Variantes de Pivote	
3.2	Análisis Teórico de Algoritmos de Búsqueda	5
	Búsqueda Binaria: Recursiva y por Rango • Búsqueda Exponencial • Búsqueda por Interpolación	
3.3	Análisis Teórico de Algoritmos de Selección	6
	Quick Select	
3.4	Resultados Experimentales	7
	Impacto del Umbral en Merge Sort • Comparativa de Pivotes en Quick Sort • Desempeño en Algoritmos de Búsqueda • Contraste: Divide y Vencerás frente a Incrementales (Tarea 1)	

3.5	Aplicación Práctica y Funcionalidades	9
4	Conclusiones	10
5	Contribución por integrante	11
5.1	Andrés Barbosa	11
5.2	Emmanuel Velásquez	11
5.3	Diego Oyarzo	11
6	anexos	12

1. Introducción

El diseño de algoritmos tiene como objetivo encontrar soluciones a problemas de manera eficiente teniendo en cuenta las limitaciones de tiempo y memoria que existen dentro de los sistemas informáticos.

Dos de los problemas más comunes en el día a día de la computación son el **ordenamiento** de datos, la **búsqueda** de información y la **selección** de elementos dentro de estructuras de datos; ambos problemas se pueden afrontar desde distintas miradas, en este caso, centrándonos en el poderoso paradigma de **Divide y Vencerás**.

El presente informe aborda estos problemas sobre una base de datos ficticia de deportistas (generada de manera procedimental) con el fin de exponer, analizar y comparar algoritmos de ordenamiento (*Merge sort* clásico y optimizado, *Quick sort* con múltiples estrategias de pivote), algoritmos de búsqueda (*Búsqueda binaria recursiva*, *exponencial* e *interpolación*) y selección (*Quick select*). Todo ello analizando su comportamiento a nivel teórico y contrastándolo mediante experimentación empírica.

2. Objetivo principal

Analizar y comparar el comportamiento de algoritmos de ordenamiento, búsqueda y selección basados en la estrategia de *Divide y Vencerás*, estableciendo una correspondencia entre los resultados teóricos y experimentales aplicados a un sistema de gestión.

2.1. Objetivos secundarios

1. Comprender el funcionamiento e implementar correctamente algoritmos con estrategia recursiva de partición.
2. Analizar empíricamente la complejidad evaluando el impacto de optimizaciones (ej: umbrales en Merge Sort) y variantes (ej: tipos de pivotes en Quick Sort).
3. Integrar los algoritmos al sistema desarrollado empleando criterios de justificación basados en la eficiencia.
4. Documentar y contrastar de manera gráfica la mejora sustancial en tiempos de ejecución frente a los algoritmos con complejidad asintótica de orden cuadrático de iteraciones previas.

3. Desarrollo

3.1. Análisis Teórico de Algoritmos de Ordenamiento

En esta sección se detallan los algoritmos de ordenamiento implementados bajo el paradigma de *Divide y Vencerás*. Se analizará su funcionamiento de manera teórica, desglosando sus pseudocódigos y estudiando su complejidad asintótica global.

3.1.1. Merge Sort: Clásico y Optimizado

El algoritmo Merge Sort basa su funcionamiento en dividir el arreglo por la mitad repetidamente hasta obtener sub-arreglos de un solo elemento, para luego fusionarlos (operación *merge*) de manera ordenada.

Algorithm 1 Merge Sort Clásico ▷ $V[beg..end]$

```

1: procedure MERGESORT( $V, beg, end$ )
2:   if  $beg < end$  then
3:      $m \leftarrow beg + \lfloor \frac{end-beg}{2} \rfloor$ 
4:     MERGESORT( $V, beg, m$ )
5:     MERGESORT( $V, m + 1, end$ )
6:     MERGE( $V, beg, m, end$ )

```

Análisis de Complejidad: Merge Sort siempre divide el arreglo en dos mitades, tomando tiempo constante $\mathcal{O}(1)$. La función MERGE combina dos sub-arreglos transicionando todos sus n elementos, lo cual toma $\mathcal{O}(n)$. La relación de recurrencia aplicable es:

$$T(n) = 2T(n/2) + \mathcal{O}(n)$$

Por el Teorema Maestro, esto siempre resulta en una complejidad temporal constante de $\mathcal{O}(n \log n)$ en el mejor, peor y caso promedio.

Variante Optimizada con Umbral (Threshold): Al delegar excesivamente al sistema la creación de árboles recursivos inmensos de llamadas y arreglos subyacentes mínimos, se penaliza el rendimiento. Algoritmos de orden cuadrático como *Insertion Sort* poseen gastos constantes diminutos para entradas limitadas. La variante optimizada introduce un límite de recursión (*UMBRAL* o *threshold*); si el tamaño de un sub-arreglo alcanza un $n \leq \text{Threshold}$, se corta la fragmentación y se despacha a la rutina de Insertion Sort, mejorando notablemente los tiempos empíricos a pesar de mantener su categoría perimetral $\mathcal{O}(n \log n)$.

3.1.2. Quick Sort y sus Variantes de Pivote

Quick Sort selecciona un elemento referencial o *pivote* y particiona el arreglo utilizando el esquema de partición original de **Lomuto**. Todos los datos menores que el pivote recaen iterativamente a su lado izquierdo, mientras que los remanentes se asientan a su derecha. Finalmente, propaga sus llamadas lógicas separadamente.

Algorithm 2 Quick Sort Recursivo ▷ Basado en partición de Lomuto

```

1: procedure QUICKSORT( $V, low, high$ )
2:   if  $low < high$  then
3:      $p \leftarrow \text{LOMUTOPARTITION}(V, low, high)$ 
4:     QUICKSORT( $V, low, p - 1$ )
5:     QUICKSORT( $V, p + 1, high$ )

```

Dependiendo de cómo se selecciona el pivote antes de particionar, el algoritmo lidia diferentemente con las anomalías de los datos (como conjuntos ya previamente ordenados). Se implementaron 4 directrices:

- **Último o Primer elemento:** Determinista e inherentemente veloz, pero susceptible y frágil frente a datos ya estructurados.
- **Elemento Aleatorio:** Invoca a los temporizadores nativos en cada paso para elegir el límite. Es probabilísticamente resiliente al *Peor Caso*.
- **Mediana de tres:** Escoge matemáticamente la mediana entre el primer, el central y el último elemento; aislando de forma consistente las desviaciones extremistas.

Análisis de Complejidad Global:

- **Mejor y Caso Promedio** $\mathcal{O}(n \log n)$: Estructura el árbol recursivo de forma simétrica si el pivote divide permanentemente el arreglo en dos particiones proporcionales.
- **Peor Caso** $\mathcal{O}(n^2)$: Ocurre si el esquema particional selecciona ininterrumpidamente un máximo o mínimo. (ej., Pivote del último elemento en arreglos totalmente ordenados).

3.2. Análisis Teórico de Algoritmos de Búsqueda

3.2.1. Búsqueda Binaria: Recursiva y por Rango

Algorithm 3 Búsqueda Binaria Recursiva ▷ $V[beg..end]$ ordenado

```

1: procedure BINSEARCHREC( $V, beg, end, x$ )
2:   if  $beg > end$  then
3:     return  $-1$ 
4:    $m \leftarrow beg + \lfloor \frac{end - beg}{2} \rfloor$ 
5:   if  $V[m] == x$  then
6:     return  $m$ 
7:   else if  $V[m] < x$  then
8:     return BINSEARCHREC( $V, m + 1, end, x$ )
9:   else
10:    return BINSEARCHREC( $V, beg, m - 1, x$ )

```

Al igual que su versión iterativa de la tarea pasada, fragmenta la zona objetivo fraccionalmente. Conlleva una complejidad temporal de $\mathcal{O}(\log n)$ para el peor y caso promedio.

Por otro lado, la variante de **Búsqueda por Rango** aplica este mismo principio logarítmico para dictar umbrales a los topes izquierdo y derecho del arreglo de matches para un solo elemento x , retornando límites masivos usando sólo una superposición asintótica de $\mathcal{O}(\log n)$.

3.2.2. Búsqueda Exponencial

La búsqueda exponencial o *crecimiento galopante* halla una delimitación asumiendo aumentos en potencias de 2 (saltando multiplicativamente índices como 1, 2, 4, 8...), para finalmente ceder la limitación precomputada a una búsqueda binaria tradicional.

Algorithm 4 Búsqueda Exponencial

```

1: procedure EXPSEARCH( $V, n, x$ )
2:   if  $V[0] == x$  then
3:     return 0
4:    $i \leftarrow 1$ 
5:   while  $i < n$  and  $V[i] \leq x$  do
6:      $i \leftarrow i * 2$ 
7:   return BINSEARCH( $V, i/2, \min(i, n - 1), x$ )

```

Posee implicaciones asombrosas a nivel temporal $\mathcal{O}(\log i)$ donde i es el índice del elemento. Conviértendola en un faro excepcionalmente raudo en grandes bases de datos.

3.2.3. Búsqueda por Interpolación

Apoya su funcionalidad interpretando numéricamente el "valor" lógico a buscar intersecándolo en una recta analítica generada entre las fronteras.

Algorithm 5 Búsqueda por Interpolación

```

1: procedure INTERPOLATIONSEARCH( $V, n, x$ )
2:    $low \leftarrow 0, high \leftarrow n - 1$ 
3:   while  $low \leq high$  and  $x \geq V[low]$  and  $x \leq V[high]$  do
4:      $pos \leftarrow low + \lfloor \frac{(x - V[low]) * (high - low)}{(V[high] - V[low])} \rfloor$ 
5:     if  $V[pos] == x$  then
6:       return  $pos$ 
7:     if  $V[pos] < x$  then  $low \leftarrow pos + 1$ 
8:     if  $V[pos] > x$  then  $high \leftarrow pos - 1$ 
9:   return  $-1$ 

```

En el mejor de los casos (distribuciones de datos absolutamente lineales) su comportamiento salta a proezas de orden minúsculo $\mathcal{O}(\log(\log n))$. Su fragilidad (Peor Caso, donde recae en $\mathcal{O}(n)$) yace en su estricta dependencia de si el conjunto del usuario de sistema expone vacíos gigantes de distribución de puntajes.

3.3. Análisis Teórico de Algoritmos de Selección**3.3.1. Quick Select**

Comparte el mismo árbol generativo de Lomuto en Quick Sort. No obstante, al momento de ser partido el hilo, deduce si el objetivo o índice k -ésimo descansó hacia la izquierda o hacia la derecha. Descartando abruptamente la vía que no posee congruencia asintótica con la ubicación del elemento deseado.

Algorithm 6 Quick Select ▷ Búsqueda indexada selectiva

```

1: procedure QUICKSELECT( $V, low, high, k$ )
2:    $p \leftarrow \text{LOMUTOPARTITION}(V, low, high)$ 
3:   if  $p == k$  then
4:     return  $V[p]$ 
5:   else if  $p > k$  then
6:     return QUICKSELECT( $V, low, p - 1, k$ )
7:   else
8:     return QUICKSELECT( $V, p + 1, high, k$ )

```

Dado que se purga un lado de los datos de forma continua y nunca se ordena el 100 % del conjunto:

- **Mejor y Caso Promedio** $\mathcal{O}(n)$: El sub-arreglo se trunca en proporción constante en la media; procesando $n + n/2 + n/4 \dots$, siendo un orden de magnitud lineal.
- **Peor Caso** $\mathcal{O}(n^2)$: Condicionado igualmente por los infortunios teóricos perimetrales de un pivoteo desbalanceado.

3.4. Resultados Experimentales

Esta sección contrasta los hallazgos teóricos de la complejidad con simulaciones obtenidas de los datasets estructurados.

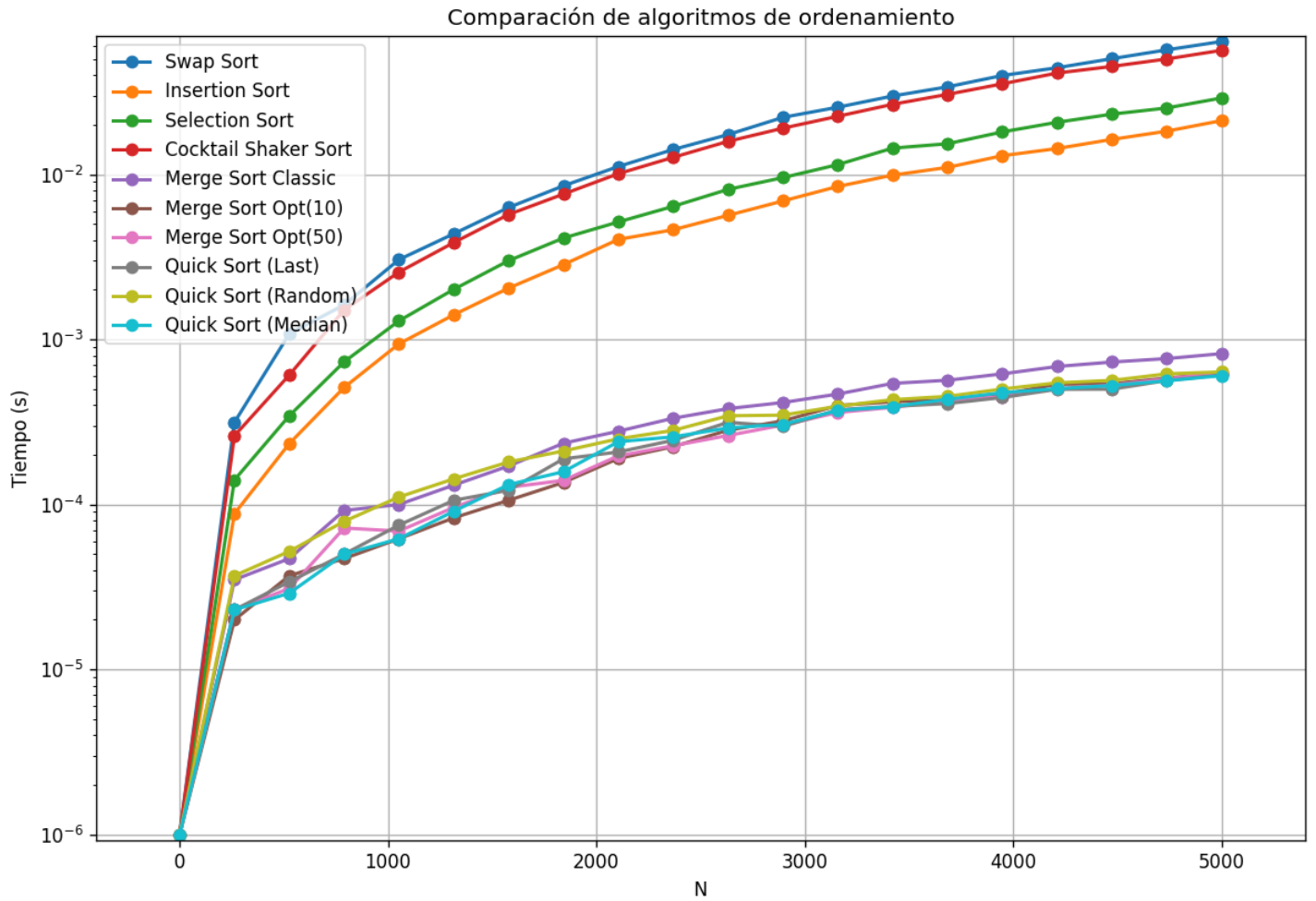


Figura 1. Comparativa logarítmica de algoritmos de ordenamiento (Divide y Vencerás vs Algoritmos de orden $\mathcal{O}(n^2)$).

3.4.1. Impacto del Umbral en Merge Sort

Basado en los resultados experimentales plasmados en la gráfica de rendimiento (escala logarítmica), se observa claramente el impacto positivo de la optimización del umbral en el algoritmo Merge Sort. La variante clásica (Merge Sort Classic) muestra tiempos de ejecución consistentemente mayores en comparación con las versiones híbridas (Merge Sort Opt(10) y Opt(50)). Al delegar el ordenamiento de subarreglos pequeños a Insertion Sort, se evitan las penalizaciones por el exceso de llamadas recursivas. Particularmente, la variante con un umbral de 10 y 50 elementos demuestra una mejora medible, ubicándose de manera estable por debajo de la curva del método clásico a medida que N aumenta, evidenciando mayor eficiencia de CPU en arreglos particionados pequeños.

3.4.2. Comparativa de Pivotes en Quick Sort

El análisis de las curvas de rendimiento revela que las diferentes estrategias de selección de pivote en Quick Sort logran estabilizar el rendimiento del algoritmo hacia su mejor y caso promedio teórico de $\mathcal{O}(n \log n)$. Las variantes de pivote último, aleatorio y mediana de tres presentan un comportamiento empírico altamente competitivo, agrupándose intrínsecamente en la zona de menor tiempo de ejecución de la gráfica. Las estrategias estocásticas y analíticas, como la mediana de tres y la aleatorización respectivamente, son supremamente eficaces previniendo el decaimiento asintótico ocasionado ante series pre-ordenadas (peor caso $\mathcal{O}(n^2)$), manteniendo sus tiempos consistentemente en el clúster inferior

global de rendimiento.

3.4.3. Desempeño en Algoritmos de Búsqueda

Experimentalmente, los algoritmos de búsqueda basados en *Divide y Vencerás* logran una reducción fenomenal en el número de operaciones frente a búsquedas lineales o exhaustivas. Búsqueda Binaria y Exponencial establecen tiempos de operación notablemente bajos garantizando topes y retornos para casos promedio y peores en el orden de $\mathcal{O}(\log n)$. De igual modo se connota que Búsqueda por Interpolación posee la capacidad condicional de eclipsar el desempeño iterativo en arreglos con distribuciones equitativas hasta $\mathcal{O}(\log(\log n))$, sin eximirse del impacto negativo ante topologías dispersas donde puede caer abruptamente a $\mathcal{O}(n)$.

3.4.4. Contraste: Divide y Vencerás frente a Incrementales (Tarea 1)

Para visualizar apropiadamente las variaciones temporales masivas que separan a las metodologías se ha recurrido a una gráfica logarítmica de ejecuciones (`sorting_graph.png`). En ella se marca una dramática brecha temporal entre los algoritmos preexistentes (Tarea 1), que mantienen una trayectoria ascendente bajo el paradigma cuadrático $\mathcal{O}(n^2)$ dictaminado por los bucles anidados constantes (Swap Sort, Selection Sort, Insertion y Cocktail). Como completa antinomia, las soluciones divisivas implementadas (Quick Sort y Merge Sort $\mathcal{O}(n \log n)$) completan permutaciones masivas en fracciones ínfimas de tiempo transaccional linealizado transversalmente, concluyendo innegablemente la superioridad escalar de las estructuras de ramificación recursivas frente al incremento computacional simple.

3.5. Aplicación Práctica y Funcionalidades

[Aquí se documentará de forma visual y teórica cómo la interfaz ha implementado las nuevas lógicas para listados de rango de puntajes, top atletas, ranking por N criterios o k-ésimo posicionado en el roster central del código fuente. Añadiendo una breve justificación sobre la selección del algoritmo para cada pantalla].

4. Conclusiones

A partir del análisis teórico y experimental realizado sobre los algoritmos de la familia *Divide y Vencerás*, y su contraste con los algoritmos iterativos clásicos, se extraen de manera general las siguientes conclusiones:

El salto de cuadrático a log-lineal

[Aquí se completará tras analizar los gráficos: Se discutirá cómo los algoritmos de ordenamiento $\mathcal{O}(n \log n)$ como Merge Sort y Quick Sort logran reducir masivamente los tiempos de ejecución frente a los de orden $\mathcal{O}(n^2)$ del proyecto anterior, justificando su coste recursivo.]

Optimización empírica: El impacto heurístico

[Aquí se completará tras analizar los gráficos: Se detallará cómo el uso de un umbral en Merge Sort u optimizar la elección del pivote en Quick Sort pueden marcar la diferencia entre un algoritmo promedio y uno altamente eficiente, demostrando que la teoría asintótica se ve directamente afectada por la heurística utilizada y la forma de los datos].

Búsquedas avanzadas

[Aquí se completará tras analizar los gráficos: Se expondrán las ventajas de la búsqueda exponencial para localizar rangos rápidamente en arreglos inmensos y cómo la búsqueda por interpolación rinde dependiendo fuertemente de la distribución de los datos].

Decisiones arquitectónicas del uso de memoria

[Aquí se completará: Breve reflexión sobre el coste espacial. Quick Sort destaca in-place frente a los arreglos auxiliares exigidos por Merge sort, destacando el impacto en entornos con recursos limitados].

5. Contribución por integrante

A continuación se detalla la contribución realizada por cada uno de los integrantes del grupo en el desarrollo del proyecto.

5.1. Andrés Barbosa

Andrés Barbosa — Actualizador y mantenedor principal del proyecto.

- Responsable de actualizar e integrar el código legado proveniente de su grupo anterior.
- Realización de pruebas y validación de las solicitudes del proyecto, asegurando la calidad y funcionalidad del código.
- Liderazgo en asignación de tareas y coordinación del equipo, facilitando la comunicación y colaboración entre los integrantes.

5.2. Emmanuel Velásquez

El estudiante **Emmanuel Velásquez** contribuyó al proyecto con las siguientes tareas:

- Actualización de documentación del repositorio acorde a Proyecto 2 asignado por la docente.
- Prototipado del informe, heredando formato anterior y adaptandolo a los requerimientos del proyecto.
- Revisión de salud de repositorio, constancia de que mantiene archivos correspondientes al día para un correcto uso de git.

5.3. Diego Oyarzo

El estudiante **Diego Oyarzo** contribuyó al proyecto con las siguientes tareas:

- Corrección de errores en análisis de algoritmos.
- Inspector de código, análisis de algoritmos y pruebas.

6. anexos

```
1  /**
2   * @brief Esquema de particion de Lomuto
3   */
4  static int lomuto_partition(Player V[], int low, int high, int pivot_type, int (*comp_f)(Player *, Player *)) {
5      int p_idx = select_pivot_index(V, low, high, pivot_type, comp_f);
6      Player pivot = V[high];
7
8      int i = low - 1;
9      for (int j = low; j < high; j++) {
10         if (comp_f(&V[j], &pivot) <= 0) {
11             i++;
12             swap_player(&V[i], &V[j]);
13         }
14     }
15     swap_player(&V[i + 1], &V[high]);
16     return i + 1;
17 }
```

Código 1. Esquema de partición Lomuto empleado como base en Quick Sort y Quick Select

■ Referencias

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical algorithms*, 3.^a ed. Addison-Wesley, 1997, ISBN: 978-0-201-89684-8. (visitado 13-11-2019).
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2.^a ed. Addison-Wesley, 1998, ISBN: 978-0-201-89685-5. (visitado 02-04-2026).
- [3] Linux Kernel Organization, *Linux Programmer's Manual: QSORT(3)*, Accedido el 12 de abril de 2026, 2024. dirección: <https://man7.org/linux/man-pages/man3/qsort.3.html>.